



Sky: Xchain Helpers

Security Review

Cantina Managed review by:

Christoph Michel, Lead Security Researcher

Mario.eth, Lead Security Researcher

October 8, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	LZForwarder uses wrong sender in quote	4
3.2	Low Risk	4
3.2.1	LZForwarder might refund the wrong payer	4
3.2.2	lzTokenFee is assumed to be 0	5
3.2.3	LZReceiver.lzReceive is not payable	5
3.3	Informational	5
3.3.1	LZReceiver does not implement full OApp interface	5

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Sky Protocol is a decentralised protocol developed around the USDS stablecoin.

From Aug 26th to Sep 1st the Cantina team conducted a review of `xchain-helpers` on commit hash `610cede5`. The team identified a total of **5** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	3	3	0
Gas Optimizations	0	0	0
Informational	1	1	0
Total	5	5	0

The Cantina Managed team reviewed Sky's `xchain-helpers` holistically on commit hash `33baf1e9` (corresponding to version tag `v1.1.0`) and concluded that all findings were addressed and no new vulnerabilities were identified.

3 Findings

3.1 Medium Risk

3.1.1 LZForwarder uses wrong sender in quote

Severity: Severity: Medium Risk

Context: LZForwarder.sol#L63

Description: The LZForwarder creates the message and calls Endpoint's quote with its own msg.sender (call it A) as the sender:

```
MessagingFee memory fee = endpoint.quote(params, msg.sender);
```

The EndpointV2.quote(_params, _sender) then uses this parameter (A) to get the nonce, send library, and sets the packet.sender to it.

Whereas, the LZForwarder's subsequent send call ends up using EndpointV2.send(..) the Endpoint's msg.sender for these actions, which is the LZForwarder contract (or contract caller of this library to be precise; call it B):

```
// in EndpointV2
// NOTE: uses its own `msg.sender` here to create the message, get the send library, etc.
(MessagingReceipt memory receipt, address _sendLibrary) = _send(msg.sender, _params);
```

This leads to the quote call using a different sender than the actual send call. If the quote underestimates the actual fee used in send, the send call can revert leading to a permanent DoS. This can happen if:

1. The contract using this library is not upgradeable.
2. The send library returns pricing that depends on a value derived from the sender, like the sender itself or the nonce.
3. The different senders could each also use a different send library, for example, one hardcoded it, the other didn't, and is therefore auto-updating to the latest one defined in the EndpointV2.

Note that the default configuration for an OApp is to always use the latest send library, meaning, even if it is not an issue in the current send library, it can happen down the road when a new one is set.

Recommendation: Consider calling quote with this as the sender instead.

```
MessagingFee memory fee = endpoint.quote(params, address(this));
```

Sky: Fixed in PR 38

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 LZForwarder might refund the wrong payer

Severity: Severity: Low Risk

Context: LZForwarder.sol#L65

Description: The LZForwarder creates the message and calls Endpoint's send with its own msg.sender as the refundAddress:

```
endpoint.send{ value: fee.nativeFee }(params, msg.sender);
```

The refund should go to the ultimate payer.

In the [lz-gov example](#), the MCD_PAUSE_PROXY calls the L1_SPARK_PROXY (with no value), which delegaterecalls into the LZCrosschainPayload using this library.

The L1_SPARK_PROXY is the payer, but the refundAddress is set to msg.sender = MCD_PAUSE_PROXY. The wrong contract is being refunded.

Recommendation: Consider adding a refundAddress parameter to LZForwarder.sendMessage and use that in the endpoint.send call.

Sky: Fixed in [PR 39](#)

Cantina Managed: Fix verified.

3.2.2 `lzTokenFee` is assumed to be 0

Severity: Severity: Low Risk

Context: [LZForwarder.sol#L65](#)

Description: The `LZForwarder` is incapable of performing a LZ token transfer as part of the fee payment. While calling `quote` with `payInLzToken: false` is expected to return `_fee.lzTokenFee == 0`, after consulting with the LayerZero team, it is not guaranteed and should generally not be relied on.

Note that the default configuration for an OApp is to always use the latest send library, meaning, even if it is not an issue in the current send library, it can happen down the road when a new one is set.

Recommendation: Consider accepting the risk or fixing the `send` library for the message sender OApp (`setSendLibrary`) so it does not automatically update to a new version that might return a non-zero LZ token fee. Alternatively, add LZ token support to the `LZForwarder` library.

Sky: Fixed in [PR 41](#)

Cantina Managed: Fix verified.

3.2.3 `LZReceiver.lzReceive` is not payable

Severity: Severity: Low Risk

Context: [LZReceiver.sol#L45](#)

Description: It's possible to encode a `msg.value` in the `LZ options` passed to `LZForwarder.send`:

```
bytes memory options = OptionsBuilder.newOptions().addExecutorLzReceiveOption(200_000, 1 ether);
```

While there's no guarantee on the executing side, the current `LZReceiver.lzReceive` function is not marked as payable and will revert if the message is delivered with a non-zero value.

Recommendation: Consider marking `lzReceive` as payable. Consider forwarding this `msg.value` to the target by using `target.functionCallWithValue(_message, msg.value)`.

Sky: Fixed in commit [80e689e4](#)

Cantina Managed: Verified.

3.3 Informational

3.3.1 `LZReceiver` does not implement full OApp interface

Severity: Severity: Informational

Context: [LZReceiver.sol#L16](#)

Description: LayerZero uses the concept of OApps paired on remote chains. The `LzReceiver` contract is such an OApp that receives messages from a hardcoded peer (using `LzForwarded`). However, the `LZReceiver` contract does not implement the standard OApp interface as it does not inherit from the OApp (or `OAppReceiver`) contract.

The intention for the `LZReceiver` seems to be a stripped-down version of `OAppReceiver` with a hardcoded peer and no owner (not inheriting `Ownable`). There are still the following differences:

1. It's unclear if `LZReceiver` should call `endpoint.setDelegate(_delegate)` to set a delegate. This depends on if an external delegate address should be able to configure the OApp on the endpoint.
2. It's missing the function `oAppVersion()` external view returns `(uint64 senderVersion, uint64 receiverVersion)`; function. Should return `(0, RECEIVER_VERSION=2)`.
3. It's missing the function `endpoint()` external view returns `(address)`; function returning the `destinationEndpoint`.

4. It's missing the function `peers(uint32 _eid)` external view returns (bytes32 peer); function. It could return `sourceAuthority` for the given `srcEid`, otherwise `address(0)`.
5. It's not emitting the event `PeerSet(uint32 eid, bytes32 peer)`; in the constructor for (srcEid, sourceAuthority).
6. It's missing the function `isComposeMsgSender(Origin calldata /*_origin*/, bytes calldata /*_message*/, address _sender)` public view virtual returns (bool) function.
7. It's missing the function `nextNonce(uint32 _srcEid, bytes32 _sender)` public view virtual returns (uint64 nonce) function.

Note that some functions might be required for the LayerZero off-chain infrastructure (executors), as indicated by this comment in `nextNonce`:

```
@dev Is required by the off-chain executor to determine the OApp expects msg execution is ordered..
```

Recommendation:

1. Determine if a delegate for the `LZReceiver` OApp should be set. Determine if they should perform any of the following actions:
 1. Have the ability to `clear/skip/nilify/burn` a message.
 2. Set/lock-in a custom receive library.
 3. Set a custom config.
2. Consult with LayerZero to determine if any of the missing functionality is required for a receive-only OApp. Also see [Custom OApp](#) as reference which states to inherit from OApp.
3. Consider implementing the missing functionality to be compatible with a standard OApp interface.

Sky: Fixed in commit [80e689e4](#)

Cantina Managed: Verified. Note that the new code is inheriting from OApp now, adding an `owner` and `send` functionality. The `peer[srcEid]` and `sourceAuthority` might become out of sync if the owner calls `setPeer`.